

Backend Fundamentals — Recap 3

Status codes, real bugs, validation, error handling, and middleware

Picking up right after Recap 2 — nodemon was already covered there.

1. 201 Created — Telling the Truth About What Happened

200 means "OK" generically. 201 specifically means "something new now exists because of this request" — the honest status for a successful POST that creates a resource.

```
app.post('/tasks', (req, res) => {
  const newTask = { id: nextId, title: req.body.title };
  tasks.push(newTask);
  res.status(201).send(newTask);
});
```

curl hides status codes by default — add `-i` to see the full response, including headers and the status line (e.g. HTTP/1.1 201 Created).

2. Real Bug: Duplicate IDs After Delete

Using `tasks.length + 1` to assign new ids breaks once deletion is possible — length shrinks when an item is removed, so a new id can collide with one that still exists. Example: 3 tasks (ids 1,2,3) → delete id 2 → length becomes 2 → next id calculated as 2+1=3 → collides with the existing task 3.

Fix: a counter that only ever increases, independent of array length.

```
let nextId = 1;

app.post('/tasks', (req, res) => {
  const newTask = { id: nextId, title: req.body.title };
  nextId++;
  tasks.push(newTask);
  res.status(201).send(newTask);
});
```

3. Validation — Missing or Empty Input

Never trust incoming data. Check it on the server before acting on it, and reject bad input clearly with **400 Bad Request** (different meaning from 404 — 400 means the client sent something wrong; 404 means the thing being looked up doesn't exist).

```
if (!req.body.title || req.body.title.trim() === '') {
  return res.status(400).send('Title is required');
}
```

Applied to both POST and PUT. When both a bad id and a bad title are present in the same request, whichever check appears first in the code wins — order of checks matters.

4. Real Bug: Crash From Wrong Data Type

Sending a number instead of a string as `title` (e.g. `{ "title": 12345 }`) passed the emptiness check — a number isn't falsy and isn't an empty string — but then crashed on `.trim()`, because numbers don't have that method.

`TypeError: req.body.title.trim is not a function`

Fix: check the type first, before calling any string-only method.

```
if (typeof req.body.title !== 'string' || req.body.title.trim() === '') {  
  return res.status(400).send('Title must be a non-empty string');  
}
```

Because of how `||` short-circuits, the type check runs first and stops immediately if it fails — `.trim()` never runs on a non-string value.

5. Bonus Edge Case: Non-Numeric IDs

Visiting a route like `/tasks/abc` turns `Number('abc')` into `NaN`. In JavaScript, `NaN` is never equal to anything — not even itself (`NaN === NaN` is `false`). So `.find()` matches nothing, and the existing 404 check handles it safely without any extra code.

6. Error Handling: What Happens When Something Slips Through

Express automatically catches errors thrown **synchronously** inside a route handler — the request gets an ugly default error page (status 500), but the server process itself keeps running; other requests are unaffected.

The important gap: errors thrown **asynchronously** (inside a callback, `.then()`, or an async function without explicit handling) are NOT automatically caught by that same safety net, and can crash the entire Node process — full downtime, not just one bad request.

A global error-handling middleware replaces the raw stack-trace page with a clean, safe response (and avoids leaking internal file paths/code to real users):

```
// last thing before app.listen  
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something went wrong on our end.');
```

```
});  
In real production systems this is only one layer — a process manager (e.g. PM2, or what Docker/Kubernetes provide) restarts a fully crashed app automatically, and logging/monitoring tools (e.g. Sentry) alert a developer instead of relying on someone watching a terminal.
```

7. Middleware — Functions Between Request and Route

Middleware is a function that runs **between** an incoming request and the route handler that finally responds to it. It can inspect or modify the request, and either let it continue (`next()`) or stop it by sending

a response itself.

Already-used examples, now named:

```
app.use(express.json()); // parses JSON body before any route sees req.body
```

```
app.use((err, req, res, next) => { ... }); // the error handler above
```

A custom logging middleware, added near the top of the file:

```
app.use((req, res, next) => {  
  console.log(`${req.method} ${req.url}`);  
  next();  
});
```

Normal middleware takes three parameters: `(req, res, next)`. Calling `next()` passes the request along the chain. Not calling it (and not sending a response) leaves the request hanging forever. The whole app is really a chain: request → middleware(s) in order → matching route → response.

8. Where the API Stands Now

- Correct status codes: 200, 201, 400, 404, 500
- No duplicate ids after deletion (nextId counter)
- Validation on POST and PUT: rejects missing, empty, and wrong-type titles
- No crash-on-bad-input bugs
- Global error handler as a safety net for the unexpected
- Custom logging middleware, and a working mental model of the middleware chain

Next up: replacing the in-memory array with a real, persistent database.